Arrays and Strings

Module-1



Dr.M. Sivakumar

Professor & Technical Trainer / CSE

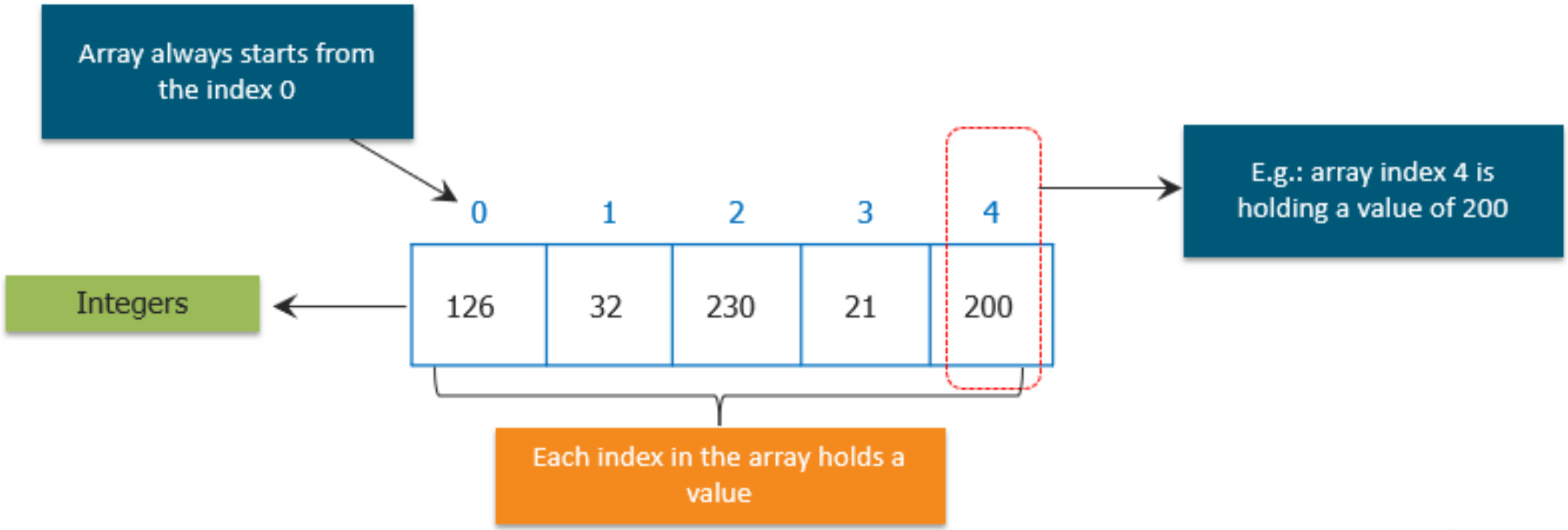
Saveetha School of Engineering, SIMATS

sivakumarm.sse@saveetha.com

+91-9500600868

Arrays

- An Array is a group of homogenous elements i.e., all elements in array have same data type.
- It stores the value on the basis of the index value.



Arrays

Array - Declaration:

The array declaration can be in any of the below forms:

```
int intArray[ ];

int[ ] intArray;

int   [ ]intArray;

int[ ] arr1, arr2; //Both arr1 and arr2 are arrays.

int [ ]arr1, arr2; //Only arr1 is an array but arr2 is not
```

Arrays

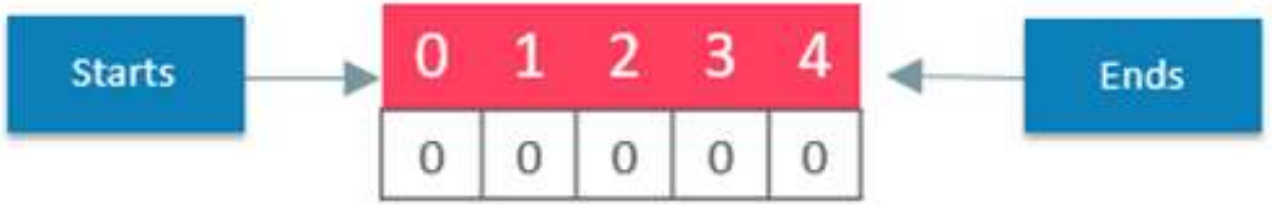
Example

data type

size of array

```
int[]a= new int[5];
```

Index has to be given in square brackets

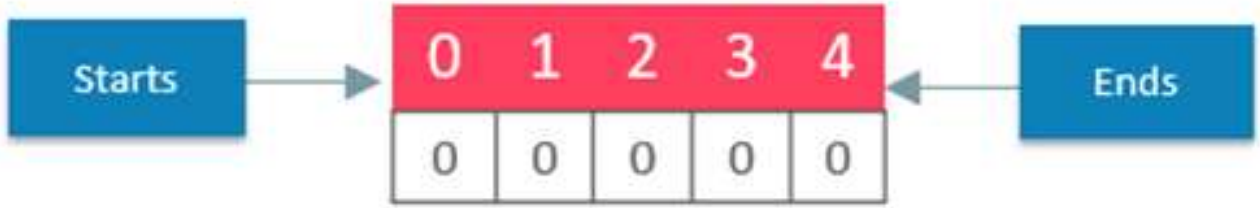


data type

size of array

```
int a[]= new int[5];
```

Index has to be given in square brackets



Arrays

Properties of an Array:

- It is an index based collection whose index starts at 0 and ends with index (n-1)
- The size of an array is fixed and cannot be changed
- Arrays are created dynamically at run-time using ‘new’ keyword
- The elements in an array are either Primitive or Object (reference) type
- It can also be used as a static field, a local variable, and a method parameter.

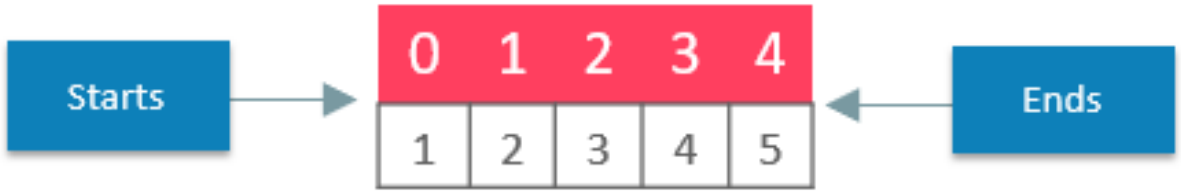
Array - Initialization

data type

size of array

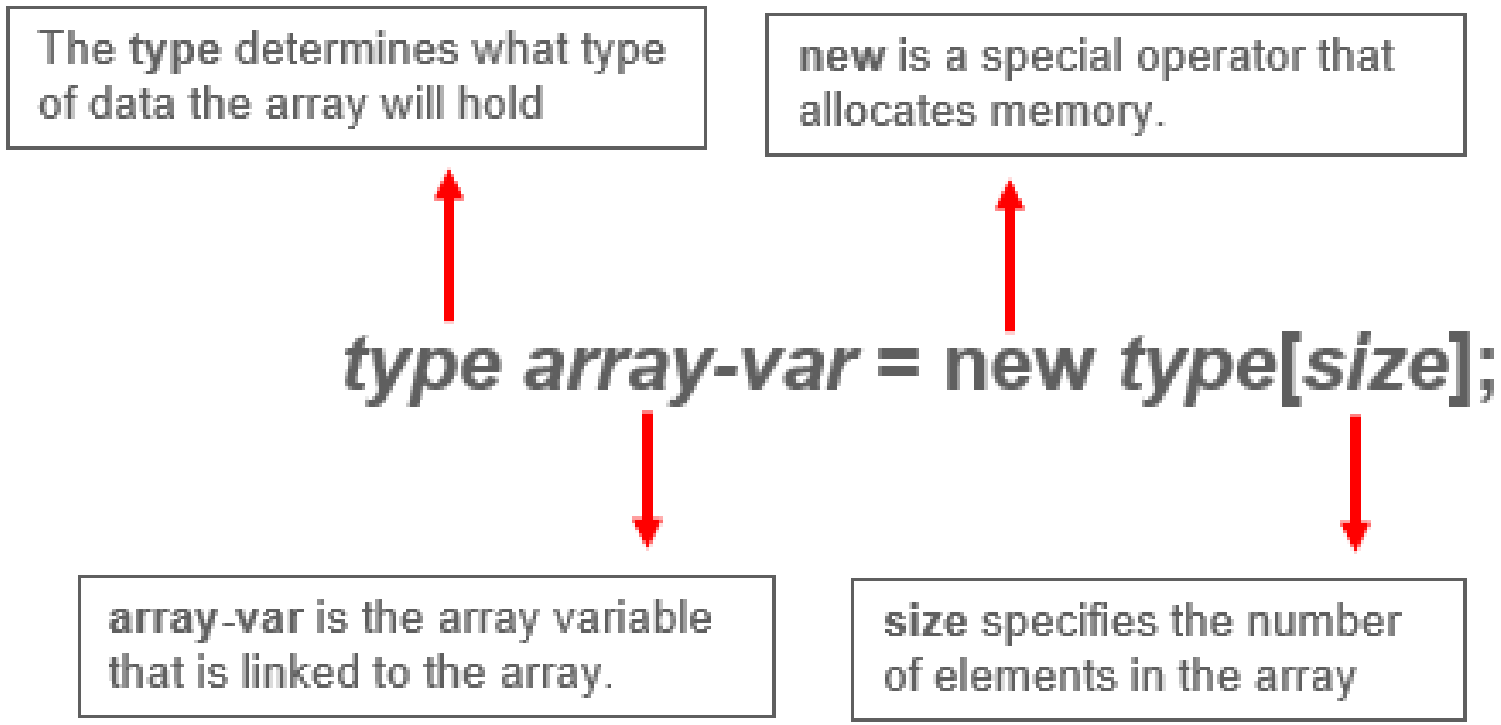
```
int[]a= new int[]{1,2,3,4,5};
```

Index has to be given in square brackets



Arrays

The general form an array is:



Array - Initialization

The **type** determines what type of data the array will hold

type var-name[] = {value1, value2, value3, value4,…};

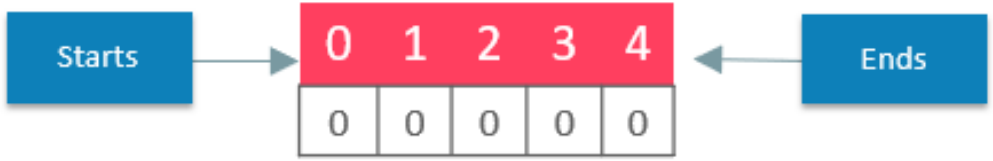
An array initializer is a list of comma-separated expressions surrounded by curly braces. The commas separate the values of the array elements

data type

size of array

```
int[]a= {1,2,3,4,5};
```

Index has to be given in square brackets



Arrays

Anonymous Array:

- ✓ Array without having any name is called Anonymous Array
- ✓ The main object of anonymous arrays is , it is created only for instant use

Syntax:

new DataType[ ]{values} ;

```
public static void print(int a[]) {
    for (int i = 0; i < a.length; i++) {
        System.out.print(a[i] + "\t");
    }

}
public static void main(String[] args) {
    //we will pass anonymous array in print() method
    Example.print(new int[]{1, 3, 2});
}
```

Arrays

Length Vs length( ):

- The ‘length’ field is used to get array size.
- The ‘length()’ method is used to get string size

```
int arr[] = {1,2,3,4};

System.out.println(arr.length);     // 4

String str = new String(“ONLINE”);

System.out.println(str.length());    // 6
```

Arrays

Example

```
class Example {

    public static void main(String[] args) {
        int marks[] = new int[5];
        //taking input from user and store in array
        Scanner ob = new Scanner(System.in);
        for (int i = 0; i < marks.length; i++) {
            marks[i] = ob.nextInt();
        }
        int sum = 0;
        //adding elements one by one
        for (int i = 0; i < marks.length; i++) {
            sum = sum + marks[i];
        }
        System.out.println("Sum of elements = " + sum);
    }
}
```

Problem-1

Program to get marks of five subjects and print the status pass or fail.

Note: If fails, print the total number of subjects failed.

Arrays

Array – Copying:

```
int[ ] x = {1, 2, 3, 4, 5};
int[ ] y = x; //don't copy array like this!
```

Three Methods:

1. Clone method

```
int[ ] x = {1, 2, 3, 4, 5}; int[] y = x.clone();
```

2.copyOf() method

```
int [ ]myarray=new int[]{1,3,4,5,6};
int size=myarray.length;
int a[ ]=Arrays.copyOf(myarray,size);
```

3.System.arraycopy() method

```
arraycopy(Object src, int s_Pos, Object dest, int d_Pos, int len)
```

Arrays

Two Dimensional Array:

- Array of arrays
- Data is stored in row and columns and we can access the record using both the row index and column index

```
int[ ][ ] multiples = new int[4][2];

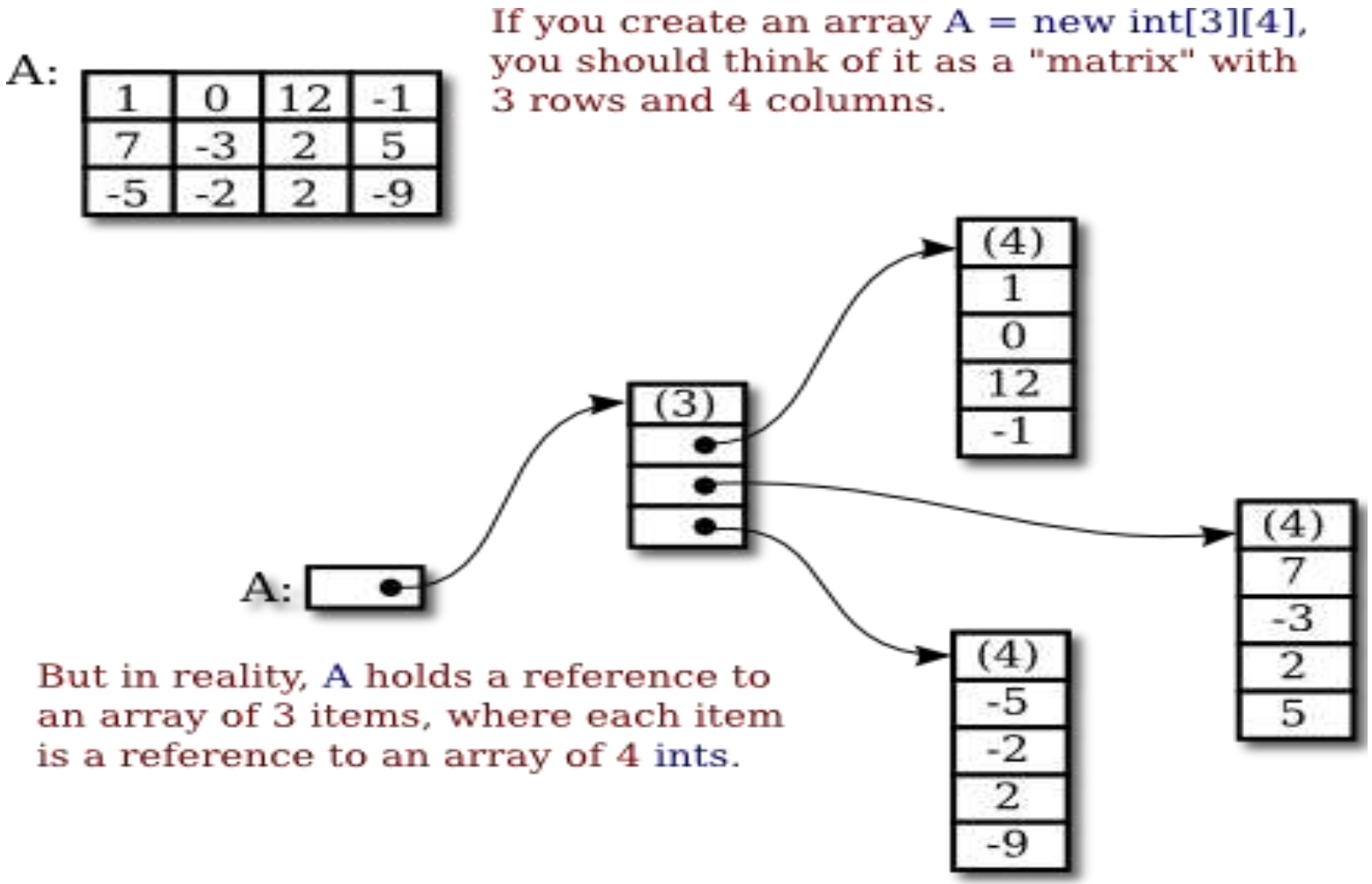
String[ ][ ] cities = new String[3][3];
```

```
int[ ][ ] wrong = new int[ ][ ];

int[ ][ ] right = new int[2][ ];
```

Arrays

Two Dimensional Array:



Arrays

Two Dimensional Array:

- When you allocate memory for a multi-dimensional array, you need only specify the memory for the first (leftmost) dimension
- We can allocate the remaining dimensions separately
- In java, the length of each array in a multidimensional array is under your control

```
int twoD[][] = new int[4][];
twoD[0] = new int[5];
twoD[1] = new int[6];
twoD[2] = new int[7];
twoD[3] = new int[8];
```

Arrays - Problems

- Program to find second largest number in an array
- Program to print the first repeated character in an array
- Program to print the last repeated character in an array
- Program to print the most repeated number in an array
- Program to print all unique elements in an array
- Program to rotate an array by ‘N’ positions
- Program to remove all duplicates in an array

Strings

- String is a sequence of characters enclosed within double quotes (" ") is known as **String**
- In java programming to store the character data we have a fundamental data type called **char**. Similarly to store the string data and to perform various operation on String data, we have three **predefined classes** they are:
  - String
  - StringBuffer
  - StringBuilder

Strings

String Class

- It is a predefined class in java.lang package can be used to handle the String.
- String class is **immutable** that means whose content can not be changed at the time of execution of program.
- **String** class object is immutable that means when we create an object of String class it never changes in the existing object.

String Class

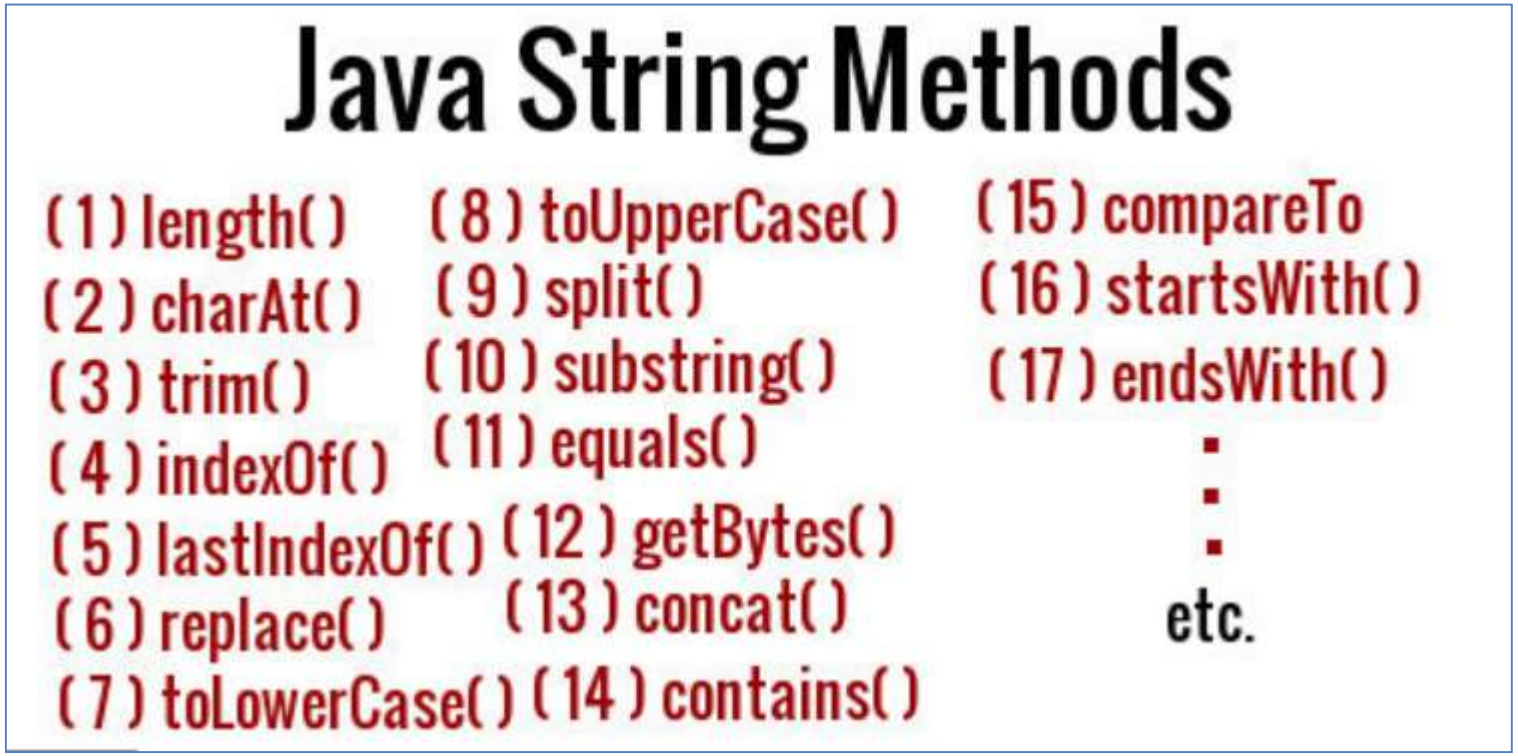
Example

```
class StringHandling
{
public static void main(String arg[])
{
String s=new String("java");
s.concat("software");
System.out.println(s);
}
}
```

Output

```
java
```

String Class - Methods



Strings

String Class - Methods

- **length():** This method is used to get the number of character of any string.

```
String s=new String("Java");
System.out.println(s.length()); // 4
```

- **charAt():** This method is used to get the character at a given index value.

```
String s=new String("Java");
System.out.println(s.charAt(2)); // 'v'
```

- **concat():** This method is used to combined two string.

```
String s1="CSE";
String s2=" Students";
System.out.println(s1.concat(s2));
```

Strings

String Class - Methods

- **subString():** This method is used to get the part of given string.

```
String s="Java is programming language";
System.out.println(s.substring(8));  // 8 is starting index
```

- **indexOf():** This method is used find the index value of given string. It always gives starting index value of first occurrence of string.

```
String s="Java is programming language";

System.out.println(s.indexOf("programming")); // 8
```

- **split():** This method is used to divide the given string into number of parts based on delimiter (special symbols like @ space ,).

```
String s="contact@skcet.co.in";

String[] s1=s.split("@"); // divide string based on @
```

Strings

- **replace():** This method is used to return a duplicate string by replacing old character with new character.

  Note: In this method data of original string will never be modify

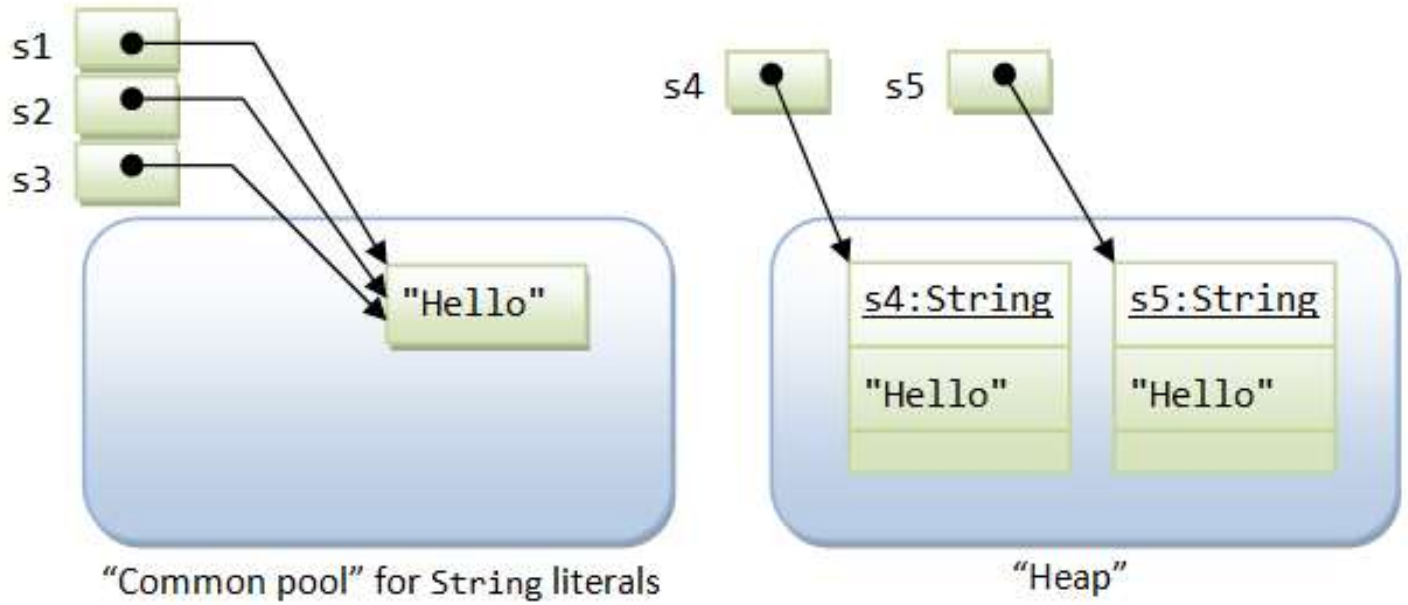
```
String s1="java";
String s2=s1.replace('j', 'k');
System.out.println(s2);
```

- **equals():** This method is used to compare two strings, It return true if strings are same otherwise return false. It is case sensitive method.

```
String s1="Hitesh";
String s2="Raddy";
String s3="Hitesh";
System.out.println("Compare String: "+s1.equals(s2)); // false
System.out.println("Compare String: "+s1.equals(s3)); // true
```

Strings - Example

```
String s1 = "Hello"; // String literal
String s2 = "Hello"; // String literal
String s3 = s1; // same reference
String s4 = new String("Hello"); // String object
String s5 = new String("Hello"); // String object
```



Strings - Example

```
String s1 = "Hello"; // String literal
String s2 = "Hello"; // String literal
String s3 = s1; // same reference
String s4 = new String("Hello"); // String object
String s5 = new String("Hello"); // String object
```

```
s1 == s1; // true, same pointer

s1 == s2; // true, s1 and s1 share storage in common pool

s1 == s3; // true, s3 is assigned same pointer as s1

s1.equals(s3); // true, same contents

s1 == s4; // false, different pointers

s1.equals(s4); // true, same contents

s4 == s5; // false, different pointers in heap

s4.equals(s5); // true, same contents
```

Strings - Problems

- Program to find the delete a particular character in a string
- Program to replace a particular character by a number ‘8’
- Program to print the last repeated character in a string
- Program to print the most repeated number in a string
- Program to replace all vowels in a string
- Program to reverse a string without using pre-defined function
- Program to compare two strings without using pre-defined function
- Program to merge two strings without using function
- Program to perform intersection of two strings